Control multi-robot basado en visión artificial con evitación de colisiones mediante campos potenciales y comunicación UDP

Estudiantes de la Cátedra de Robótica I [2025-2026]
Escuela Superior Politécnica de Chimborazo, Riobamba, Ecuador.

***Resumen*—Este trabajo presenta un sistema de control centralizado para la coordinación de *tres* robots móviles diferenciales en un área compartida, con el objetivo de alcanzar puntos de destino sin colisionar entre sí ni con los límites del entorno, se propone la arquitectura en la cual se emplea una cámara cenital conectada a una Raspberry Pi para transmitir video en tiempo real mediante *streaming* MJPEG, mientras que una PC ejecuta el procesamiento de visión por computador y el control de navegación. La localización se obtiene mediante detección de marcadores ArUco, estimando la pose planar (x, y, ψ) de cada robot; para mapear de píxeles a coordenadas métricas se utiliza una homografía calculada a partir de marcadores en las esquinas del *workspace*, reforzada con mecanismos de estabilización (histéresis y filtrado) para reducir el *jitter*. Adicionalmente, se incorpora una corrección geométrica por paralaje basada en la estimación de la pose de la cámara y en la altura del marcador del robot. El control de movimiento combina seguimiento de objetivo con evitación de colisiones mediante un campo potencial: un término atractivo hacia la meta y términos repulsivos ante la proximidad de otros robots y de paredes, generando una dirección resultante que se convierte en comandos diferenciales enviados por UDP a cada ESP32, los cuales implementan seguridad por *timeout* y rampa PWM. Los resultados experimentales evidencian operación estable en tiempo real, navegación efectiva hacia objetivos y reducción de colisiones en escenarios con trayectorias cruzadas; sin embargo, el desempeño depende de la visibilidad de los marcadores y de la calibración intrínseca de la cámara. Como trabajo futuro se plantea mejorar la robustez frente a oclusiones, completar la calibración óptica, e integrar estrategias de planificación global para minimizar mínimos locales de los campos potenciales.**

***Index Terms*—multi-robot, ArUco, homografía, campos potenciales, evitación de colisiones, UDP, ESP32, Raspberry Pi, visión artificial.**

I. Arquitectura del sistema

La solución propuesta se estructura como un sistema *centralizado por visión*, donde una cámara cenital observa simultáneamente a los tres robots y una PC ejecuta la estimación de pose y el control de navegación. La arquitectura completa integra tres bloques principales: (i) adquisición y *streaming* de video, (ii) procesamiento de visión y control multi-robot, y (iii) actuación distribuida en cada robot mediante ESP32 y un driver TB6612FNG. En la Fig. 1 se resume el flujo de datos y el lazo de realimentación.

I-A. Diagrama general y flujo de operación

El lazo de control opera de la siguiente forma:

1. **Raspberry Pi (cámara + servidor MJPEG):** captura video y lo transmite por la red local mediante *streaming* MJPEG (Flask), minimizando latencia al mantener únicamente el último *frame* disponible.
2. **PC (visión + control + UI):** recibe el flujo MJPEG, detecta marcadores ArUco, estima la pose planar (x, y, ψ) de cada robot en coordenadas métricas y calcula comandos diferenciales (izquierda/derecha) según el objetivo asignado y la lógica de evitación de colisiones.
3. **Robots (ESP32 + TB6612FNG + motores):** cada robot recibe comandos UDP y genera PWM para los motores. Se implementa un *failsafe* por *timeout* y una rampa de aceleración para suavizar cambios bruscos.

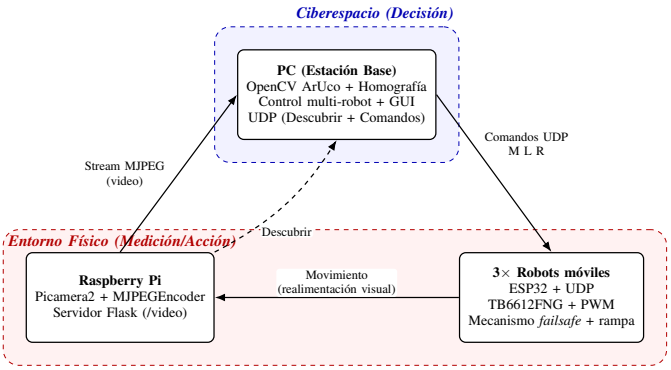

Figura 1. Arquitectura del sistema organizada por capas: Entorno Físico (OT) y Ciberespacio (IT).

I-B. Topología de red y comunicación

Todos los dispositivos se conectan a una red Wi-Fi local (router o punto de acceso), que habilita:

- **Transporte del video:** Raspberry Pi $\rightarrow$ PC vía HTTP/MJPEG.
- **Control y coordinación:** PC $\rightarrow$ Robots vía UDP (comandos) y UDP (descubrimiento).

Para simplificar la puesta en marcha, el sistema implementa un mecanismo de *descubrimiento* por broadcast: la PC envía periódicamente el mensaje `DESCUBRIR_ROBOTS` y cada robot responde con su identificación y puerto de comandos. Esto evita configurar manualmente las IP de los robots, y permite detectar desconexiones mediante temporización (*stale time*).

Cuadro I
PUERTOS Y MENSAJES PRINCIPALES EN LA COMUNICACIÓN.

Función	Puerto	Mensaje (ejemplo)
Descubrir (broadcast)	37030/UDP	`DESCUBRIR_ROBOTS`
Descubrir (respuesta)	37030/UDP	`ROBOT_AQUI ID=1 CMDPORT=44444`
Comando motores	44444/UDP	`M {-100..100} {-100..100}`
Stream de video	5000/TCP	`http://<IP_RPI>:5000/video`

I-C. Workspace y marcadores fiduciarios

El *workspace* se define mediante cuatro marcadores ArUco ubicados en las esquinas del área de trabajo, que actúan como referencias geométricas para transformar coordenadas de imagen a coordenadas métricas:

- **Esquinas del área (referencia):** IDs 4, 5, 6 y 7.
- **Robots (identificación y pose):** IDs 1, 2 y 3 (uno por robot).

Se adopta un sistema de coordenadas 2D en metros donde las esquinas del área se mapean como:

$$\begin{aligned} (4) \to (0,0), \quad (5) \to (W,0), \\ (6) \to (W,H), \quad (7) \to (0,H) \end{aligned} \tag{1}$$

Donde W y H representan el ancho y alto reales del *workspace*. Con estas correspondencias se calcula una homografía H que permite convertir puntos detectados en la imagen (u,v) a coordenadas en el plano del suelo (x,y). Para reducir inestabilidades por vibración u oclusiones parciales, la estimación de las esquinas del área se refuerza con histéresis y filtrado temporal, manteniendo un H válido durante intervalos cortos aun cuando no se observen las cuatro esquinas en un mismo *frame*.

Finalmente, dado que los marcadores de los robots pueden ubicarse a una altura h sobre el suelo (por ejemplo, sobre el chasis), se incorpora una corrección por paralaje basada en la estimación de la pose de la cámara y en la altura del marcador, con el objetivo de mejorar la coherencia métrica de la posición estimada cuando la cámara no es perfectamente ortogonal al plano del suelo.

II. METODOLOGÍA

Esta sección describe el procedimiento de implementación del sistema completo, organizado en seis etapas: (A) adquisición y *streaming* de video en Raspberry Pi, (B) detección de marcadores ArUco en PC, (C) transformación imagen–mundo mediante homografía, (D) estabilización del *workspace* con histéresis fuerte, (E) corrección por paralaje a partir de pose de cámara y altura del marcador, y (F) comunicación y descubrimiento UDP para control distribuido. Las Fig. 2 y 3 resumen el flujo global y el bloque geométrico de localización.

II-A. Adquisición y streaming *de video en Raspberry Pi*

La cámara se configura en modo video utilizando `Picamera2` y se emplea `MJPEGEncoder` para codificar directamente a JPEG dentro del *pipeline* de libcamera, evitando recodificación manual en Python. El servidor

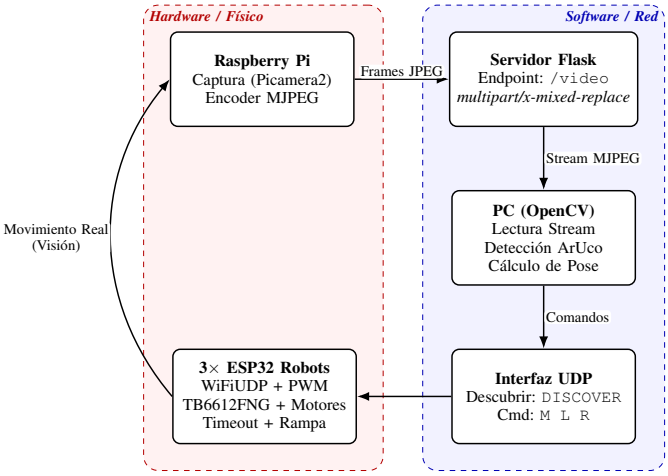

Figura 2. Flujo metodológico del sistema: adquisición, visión, control y actuación distribuida.

Flask publica el endpoint `/video` usando el tipo MIME `multipart/x-mixed-replace`, donde cada *frame* se envía como una parte JPEG independiente. Para minimizar la latencia, el *buffer* mantiene únicamente el último *frame* recibido (clase `StreamingOutput`) y el generador MJPEG no introduce `sleep`, esperando por condición a nuevos bytes JPEG.

Cuadro II
PARÁMETROS PRINCIPALES DEL *streaming* MJPEG (RASPBERRY PI).

Parámetro	Símbolo / Campo	Valor utilizado
Resolución	`WIDTH×HEIGHT`	1280 × 720
Frecuencia de cuadros	`FPS`	30
Calidad de compresión	`ENC_QUALITY`	`LOW`
Tamaño de *buffer*	`buffer_count`	3
Cola interna	`queue`	`False`
Rotación	`Transform(rotation)`	0°
Puerto HTTP	`PORT`	5000

Criterio de diseño: una calidad `LOW` y `queue=False` reducen el retardo acumulado (menos cuadros "en cola"), mientras que `buffer_count` se ajusta para evitar inestabilidad por pérdida de *frames* en hardware limitado.

II-B. Detección ArUco y preprocesamiento en PC

La PC consume el *stream* MJPEG mediante `cv2.VideoCapture(URL)` y procesa cada *frame* con OpenCV. Se utiliza el diccionario `DICT_4X4_50`, suficiente para el conjunto de IDs requeridos (1–7). Para aumentar la robustez en condiciones reales (variación de iluminación, desenfoque y vibración), se aplican dos estrategias:

- **Realce de bordes:** se aplica un *kernel* de afilado (`filter2D`) antes de la detección para reforzar los contornos del patrón.
- **Ajuste fino del detector:** se habilita refinamiento subpíxel de esquinas y umbrales adaptativos amplios para tolerar distintos contrastes.

Cuadro III
PARÁMETROS RELEVANTES DEL DETECTOR ARUCO (PC).

Bloque	Parámetro	Valor
Geometría	`minMarkerPerimeterRate`	0,003
Geometría	`polygonalApproxAccuracyRate`	0,06
Aruco3 (si disponible)	`useAruco3Detection`	`True`
Aruco3 (si disponible)	`minSideLengthCanonicalImg`	16
Proximidad	`minMarkerDistanceRate`	0,005
Refinamiento	`cornerRefinementMethod`	`SUBPIX`
Refinamiento	`cornerRefinementWinSize`	5
Refinamiento	`cornerRefinementMaxIterations`	50
Refinamiento	`cornerRefinementMinAccuracy`	0,01
Umbral adaptativo	`adaptiveThreshWinSizeMin`	5
Umbral adaptativo	`adaptiveThreshWinSizeMax`	45
Umbral adaptativo	`adaptiveThreshWinSizeStep`	10
Umbral adaptativo	`adaptiveThreshConstant`	7

II-C. *Homografía para transformación imagen–mundo*

Para expresar posiciones en metros dentro del *workspace*, se calcula una homografía H a partir de los centros de los marcadores de esquina (IDs 4–7). Sea $\mathbf{p}_i = [u_i \; v_i \; 1]^T$ el centro del marcador i en píxeles, y $\mathbf{P}_i = [x_i \; y_i \; 1]^T$ su correspondencia en el plano del mundo (metros). La homografía cumple:

$$\lambda \, \mathbf{P} = H \, \mathbf{p}, \tag{2}$$

donde λ es un factor de escala. En la implementación, se utiliza `cv2.getPerspectiveTransform(img_pts, world_pts)` con las cuatro correspondencias:

$$\begin{aligned} (4) \to (0,0), \quad (5) \to (W,0), \\ (6) \to (W,H), \quad (7) \to (0,H) \end{aligned} \tag{3}$$

Luego, cualquier punto (centro o esquina del marcador del robot) se transforma mediante `cv2.perspectiveTransform` para obtener (x, y).

II-D. *Estabilización del* workspace *mediante histéresis fuerte*

En pruebas reales, pequeñas vibraciones o variaciones de iluminación inducen *jitter* en las esquinas detectadas, lo cual provoca titileo del polígono del *workspace* y saltos en la homografía. Para mitigar este fenómeno se aplica una histéresis fuerte con memoria de centros filtrados `ws_center_filt`:

- **Umbral de anclaje (px):** si el centro detectado se mueve menos que un umbral, se *ignora* la actualización (se mantiene el valor anterior).
- **Actualización suave:** si el movimiento supera el umbral (cambio intencional), se actualiza con un promedio exponencial (EMA) para evitar saltos bruscos.
- **Hold temporal:** si algún marcador de esquina no aparece en un *frame*, se conserva el último valor válido por un tiempo `homografia_retencion_s`.

Gating anti-saltos: cuando existe una homografía previa H_{old}, se compara con la nueva H_{new} normalizando por el término $(3, 3)$ y calculando una norma de diferencia. Si el salto excede el umbral, se descarta la actualización para evitar cambios espurios.

Cuadro IV
PARÁMETROS DE ESTABILIZACIÓN DEL *workspace* Y HOMOGRAFÍA.

Elemento	Parámetro	Valor
Histéresis (esquinas)	`UMBRAL_PIXELES`	15 px
Suavizado esquinas	`WS_ALPHA`	0,8
Retención de H	`homografia_retencion_s`	1,2 s
Suavizado de H	`H_ALPHA`	0,95
Gating anti-salto	$\|A - B\|$ (norma)	$< 0{,}8$

II-E. *Corrección por paralaje (altura del marcador)*

Dado que el marcador ArUco del robot puede estar a una altura h sobre el suelo, la proyección de la cámara genera un error lateral (paralaje) si se asume que todo ocurre en $z = 0$. Para corregirlo, primero se estima la pose de la cámara usando los cuatro puntos del *workspace* en el plano $z = 0$ mediante `solvePnP` con la bandera `SOLVEPNP_IPPE`, adecuada para puntos coplanares. Si R y t representan la rotación y traslación, la posición de la cámara en coordenadas del mundo es:

$$\mathbf{C} = -R^T \, t. \tag{4}$$

Luego, si $\mathbf{P}_0 = (x_0, y_0)$ es el punto estimado por homografía sobre el suelo ($z = 0$) y $\mathbf{C}_{xy} = (C_x, C_y)$, la corrección para un objeto a altura h (con $C_z > h$) se implementa como:

$$\mathbf{P}_h = \mathbf{C}_{xy} + \left(\frac{C_z - h}{C_z} \right) (\mathbf{P}_0 - \mathbf{C}_{xy}) \, . \tag{5}$$

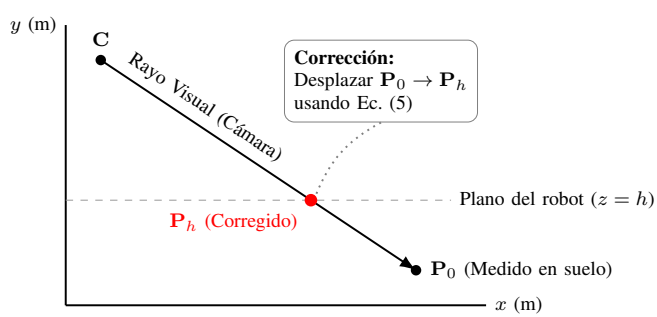

Figura 3. Geometría de la corrección: el rayo visual intersecta el suelo en $\mathbf{P}_0$, pero el marcador real está en el plano $z = h$ ($\mathbf{P}_h$).

II-F. *Comunicación y descubrimiento UDP*

La coordinación se realiza por UDP para minimizar sobrecarga de comunicación y permitir control en tiempo real. Se implementan dos canales:

1. **Descubrir (broadcast):** la PC emite periódicamente `DESCUBRIR_ROBOTS` al puerto `37030`. Cada robot responde con `ROBOT_AQUI ID=<id>CMDPORT=44444`, permitiendo asociar IP y puerto al identificador lógico del robot.
2. **Comandos de motor:** la PC envía a cada robot el mensaje `M L R`, donde L y R son porcentajes en $[-100, 100]$ para las ruedas izquierda y derecha.

Cuadro V
TIEMPOS DE COMUNICACIÓN Y CONTROL (VALORES IMPLEMENTADOS).

Bloque	**Parámetro**	**Valor**
Control PC → robots	`CMD_RATE_HZ`	12 Hz
Descubrir (PC)	`Descubrir_INTERVALO_S`	0,7 s
Robot activo (PC)	`ROBOT_ESTADO_OBSOLETO_S`	3,0 s
Timeout de comando (robot)	`COMANDO_TIMEOUT_MS`	500 ms
Anuncio periódico (robot)	`ultAnuncio`	900 ms

Seguridad en el actuador: cada ESP32 aplica *failsafe* por *timeout* (si no recibe comando en 500 ms, detiene motores) y utiliza una rampa de PWM para limitar aceleraciones, mejorando estabilidad mecánica y reduciendo derrapes.

II-G. Síntesis del procedimiento de ejecución

En operación normal, el sistema sigue el ciclo:

1. Inicializar *streaming* MJPEG en Raspberry Pi y verificar acceso desde la PC.
2. En PC, capturar *frames*, detectar ArUcos, estabilizar esquinas (IDs 4–7) y actualizar homografía H.
3. Estimar pose de robots (IDs 1–3): transformar a mundo, corregir paralaje y filtrar suavemente la pose.
4. Calcular control para cada robot: navegación hacia objetivo y evitación (términos atractivos/repulsivos) y enviar `M L R` por UDP.
5. En cada ESP32: aplicar rampa PWM, vigilar *timeout* y responder *Descubrir*.

III. ESTRATEGIA DE CONTROL Y EVITACIÓN DE COLISIONES

En esta sección se detalla la lógica de control implementada para cada robot móvil diferencial, incluyendo el seguimiento de objetivos y la evitación de colisiones entre robots y con las paredes del *workspace*. La estrategia combina: (i) un controlador direccional basado en error angular (PD) y avance proporcional a la distancia, (ii) una máquina de estados por robot para asegurar transiciones estables, y (iii) un campo potencial artificial con términos atractivos y repulsivos.

III-A. Modelo de control (cinemático) y señales de mando

Cada robot $i \in \{1, 2, 3\}$ se describe por su pose planar

$$\mathbf{s}_i = \begin{bmatrix} x_i & y_i & \psi_i \end{bmatrix}^T, \tag{6}$$

estimada por visión. El objetivo asignado se define como $\mathbf{g}_i = [x_{g,i} \;\; y_{g,i}]^T$. La salida del controlador se implementa como comandos diferenciales normalizados en porcentaje:

$$\mathbf{u}_i = \begin{bmatrix} L_i & R_i \end{bmatrix}^T, \qquad L_i, R_i \in [-100, 100], \tag{7}$$

donde L_i y R_i representan el *duty* deseado (en %) para las ruedas izquierda y derecha, enviados por UDP como `M L R`.

En la PC se calcula una componente lineal v (en % de velocidad) y una componente angular w (en % de giro), que se mezclan como:

$$L_i = \mathrm{sat}(v_i - w_i), \qquad R_i = \mathrm{sat}(v_i + w_i), \tag{8}$$

donde $\mathrm{sat}(\cdot)$ satura a $[-100, 100]$. Esta formulación coincide con el mezclado diferencial empleado en la implementación.

III-B. Control PD direccional (giro) y avance proporcional (distancia)

Dado un *heading* deseado $\psi_{d,i}$, el error angular se calcula con envoltura a $(-\pi, \pi]$:

$$e_{\psi,i} = \mathrm{wrap}_\pi(\psi_{d,i} - \psi_i). \tag{9}$$

El mando de giro se calcula con un PD discreto:

$$w_i = \mathrm{sat}\left(K_{p,\psi}\, e_{\psi,i} + K_{d,\psi}\, \frac{e_{\psi,i} - e_{\psi,i}[k-1]}{\Delta t} \right), \tag{10}$$

donde $\Delta t \approx 1/\texttt{CMD_RATE_HZ}$ es el periodo de control y $K_{d,\psi}$ actúa como amortiguamiento para reducir sobrepaso y oscilaciones.

La componente lineal se define proporcional a la distancia al objetivo y se limita por una velocidad máxima:

$$d_i = \left\| \mathbf{g}_i - \begin{bmatrix} x_i \\ y_i \end{bmatrix} \right\|_2, \qquad v_i = \mathrm{sat}_{[0, v_{\text{máx}}]}(K_v\, \alpha_d(d_i)\, \alpha_\psi(e_{\psi,i})), \tag{11}$$

donde $\alpha_d(\cdot)$ es un factor de avance por distancia (p.ej., saturado) y $\alpha_\psi(\cdot)$ atenúa el avance cuando el robot no está alineado (p.ej., basado en $\cos(e_{\psi,i})$ fuera de la fase de orientación). En la implementación se utilizan umbrales y factores de alineación distintos según el modo de navegación (Sección III-C).

Cuadro VI
PARÁMETROS REPRESENTATIVOS DEL CONTROLADOR (VALORES POR DEFECTO DEL SISTEMA).

Parámetro	**Símbolo**	**Código**	**Valor**
Ganancia lineal	K_v	`k_lin_pct_per_m`	70.0
Ganancia proporcional angular	$K_{p,\psi}$	`k_ang_pct_per_rad`	10.0
Ganancia derivativa angular	$K_{d,\psi}$	`k_ang_d_pct`	3.5
Saturación de mando	$v_{\text{máx}}$	`vmax_pct`	40.0
Tolerancia de llegada	d_{tol}	`dist_tolerance`	0.02 m
Frecuencia de control	f_c	`CMD_RATE_HZ`	12 Hz

III-C. Máquina de estados por robot

Para evitar comportamientos indeseados (p.ej., reorientaciones repetitivas tras una evasión), se emplea una máquina de estados finitos por robot con cuatro modos:

- **REPOSO:** sin objetivo asignado. Los motores se fijan a cero y se reinicia la memoria del error.
- **ORIENT:** orientación inicial. El robot gira sobre su eje hasta que $|e_{\psi,i}| \le \theta_{\text{align}}$, sin avance lineal ($v_i = 0$).
- **AVANCE:** seguimiento del objetivo con avance y giro coordinados.
- **AVOID:** evasión activa cuando la repulsión es significativa. Al volver a zona segura se retorna a **AVANCE** (sin regresar a **ORIENT**), evitando bucles.

La Fig. 4 resume las transiciones principales.

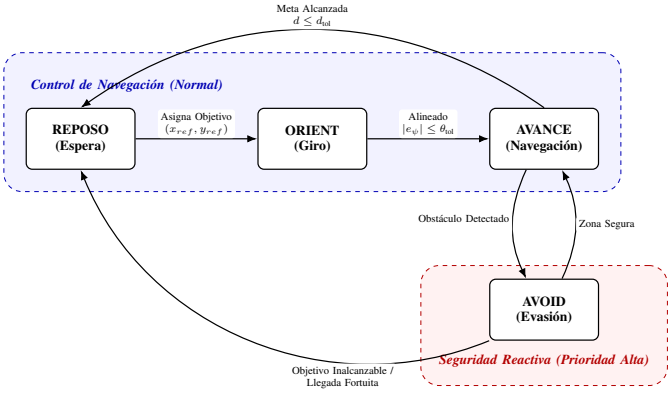

Figura 4. Máquina de estados del robot dividida en zonas de operación normal y evasión reactiva.

III-D. Evitación de colisiones mediante campos potenciales

La evitación se implementa con un campo potencial artificial en el plano (x, y). Para cada robot i, se calcula un vector de atracción hacia el objetivo y un vector de repulsión por otros robots y paredes.

III-D1. Término atractivo: Sea $\mathbf{r}_i = [x_i \; y_i]^T$ la posición del robot. El vector hacia el objetivo es:

$$\mathbf{v}_{g,i} = \mathbf{g}_i - \mathbf{r}_i, \qquad \mathbf{u}_{g,i} = \begin{cases} \dfrac{\mathbf{v}_{g,i}}{\|\mathbf{v}_{g,i}\|_2}, & \|\mathbf{v}_{g,i}\|_2 > \varepsilon, \\ \mathbf{0}, & \text{caso contrario}, \end{cases} \tag{12}$$

donde ε evita división por cero.

III-D2. Términos repulsivos: robots y paredes: Para cada robot vecino $j \neq i$ dentro de un radio de influencia d_0:

$$d_{ij} = \|\mathbf{r}_i - \mathbf{r}_j\|_2,$$
$$\mathbf{u}_{r,i}^{(j)} = \begin{cases} k_r \left(\dfrac{1}{d_{ij}} - \dfrac{1}{d_0} \right) \dfrac{(\mathbf{r}_i - \mathbf{r}_j)}{d_{ij}^2}, & 0 < d_{ij} < d_0, \\ \mathbf{0}, & d_{ij} \geq d_0. \end{cases} \tag{13}$$

donde k_r es la ganancia repulsiva. Para las paredes se emplea una zona de influencia d_w (en metros) que penaliza la cercanía a los límites del rectángulo $[0, W] \times [0, H]$:

$$\mathbf{u}_{r,i}^{(\text{walls})} = \begin{bmatrix} \phi(x_i; 0, d_w) - \phi(W - x_i; 0, d_w) \\ \phi(y_i; 0, d_w) - \phi(H - y_i; 0, d_w) \end{bmatrix},$$

$$\phi(z; 0, d_w) = \begin{cases} k_w \left(\dfrac{1}{\text{máx}(z, \epsilon)} - \dfrac{1}{d_w} \right), & 0 < z < d_w, \\ 0, & z \geq d_w. \end{cases} \tag{14}$$

con k_w proporcional a k_r y ϵ pequeño para estabilidad numérica.

El vector de repulsión total se obtiene por suma:

$$\mathbf{u}_{r,i} = \sum_{j \neq i} \mathbf{u}_{r,i}^{(j)} + \mathbf{u}_{r,i}^{(\text{walls})}. \tag{15}$$

Para evitar saturación y comportamientos bruscos, se limita la magnitud:

$$\mathbf{u}_{r,i} \leftarrow \begin{cases} \mathbf{u}_{r,i} \dfrac{u_{r,\text{máx}}}{\|\mathbf{u}_{r,i}\|_2}, & \|\mathbf{u}_{r,i}\|_2 > u_{r,\text{máx}}, \\ \mathbf{u}_{r,i}, & \text{caso contrario}. \end{cases} \tag{16}$$

Finalmente, la dirección resultante de navegación se calcula como:

$$\mathbf{u}_i = \mathbf{u}_{g,i} + \mathbf{u}_{r,i}. \tag{17}$$

En modo **AVANCE/AVOID**, el *heading* deseado se define por la resultante:

$$\psi_{d,i} = \text{atan2}(u_{i,y}, u_{i,x}), \tag{18}$$

mientras que en **ORIENT** se usa el *heading* hacia el objetivo puro (solo $\mathbf{u}_{g,i}$), garantizando una orientación inicial consistente.

Cuadro VII
PARÁMETROS DEL CAMPO POTENCIAL Y ZONAS DE SEGURIDAD (VALORES POR DEFECTO DEL SISTEMA).

Elemento	**Símbolo**	**Código**	**Valor**
Radio de evitación robots	d_0	`avoid_radiO`	0.20 m
Ganancia repulsiva robots	k_r	`k_rep`	0.70
Zona repulsión paredes	d_w	`wall_d0`	0.025 m
Ganancia repulsión paredes	k_w	`wall_k`	0,6 k_r
Límite repulsión	$u_{r,\text{máx}}$	`MAX_REPULSION`	1.0
Criterio zona segura	—	`norm_rep <0.15`	umbral

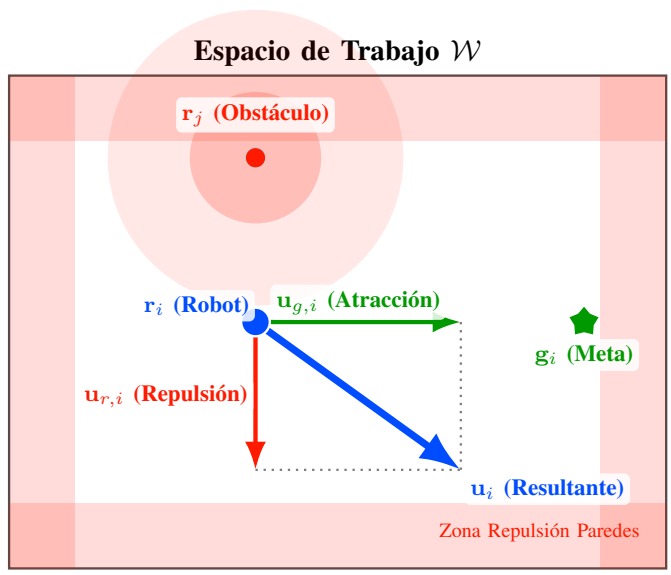

Figura 5. Representación visual de la navegación: la repulsión del obstáculo (rojo) y la atracción de la meta (verde) generan una trayectoria resultante (azul) libre de colisiones.

III-E. Detalles prácticos: zona muerta y saturaciones

Para vencer fricción estática y zonas muertas del tren motriz, se aplica un umbral mínimo de PWM en PC antes de enviar comandos:

$$
\begin{aligned}
L_i &\leftarrow \operatorname{sgn}(L_i)\ \text{máx}(|L_i|, \mathrm{PWM}_{\text{mín}}),\\
R_i &\leftarrow \operatorname{sgn}(R_i)\ \text{máx}(|R_i|, \mathrm{PWM}_{\text{mín}}).
\end{aligned} \tag{19}
$$

Cuando $|L_i|, |R_i| > 1$ y $\mathrm{PWM}_{\text{mín}}$ es un valor pequeño, y se emplean márgenes angulares diferentes según el modo (alineación estricta en **ORIENT** y tolerancia mayor en **AVANCE/AVOID**).

III-F. Seguridad y robustez en el robot (ESP32)

Cada robot ejecuta firmware con tres mecanismos de robustez:

1. **Failsafe por *timeout*:** si no se recibe un comando en un tiempo T_{out}, los motores se detienen automáticamente.
2. **Rampa de PWM:** los mandos objetivo (L_i, R_i) se alcanzan gradualmente con un incremento por paso, evitando cambios bruscos.
3. **Descubrir y anuncio periódico:** además de responder a broadcast, el robot anuncia su presencia periódicamente para facilitar reconexiones.

Cuadro VIII
PARÁMETROS RELEVANTES DEL FIRMWARE ESP32 PARA ACTUACIÓN SEGURA.

Elemento	Símbolo	Código	Valor
Frecuencia PWM	f_{PWM}	`PWM_FREQ`	20 kHz
Resolución PWM	N	`PWM_RES`	10 bits
Paso de rampa	Δ	`RAMP_STEP`	12
Retardo rampa	T_r	`RAMP_DELAY`	2 ms
Timeout de comando	T_{out}	`COMANDO_TIMEOUT_MS`	500 ms

IV. IMPLEMENTACIÓN (HARDWARE Y SOFTWARE)

Esta sección describe los componentes físicos y de software utilizados para materializar el sistema multi-robot, así como la interfaz gráfica empleada para supervisión y ajuste en tiempo real. La Fig. 6 resume la correspondencia hardware–software por dispositivo.

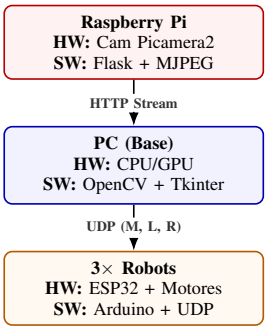

Figura 6. Mapa de implementación hardware y software por dispositivo.

IV-A. Hardware

IV-A1. Estación de procesamiento (PC): La PC ejecuta el procesamiento de visión por computador y el control multi-robot. Sus funciones principales son: lectura del *stream* MJPEG, detección ArUco, transformación geométrica a coordenadas métricas, ejecución del control (incluida la evitación de colisiones), y envío de comandos UDP a cada robot. La PC también ejecuta la interfaz gráfica para supervisión y ajuste de parámetros.

IV-A2. Nodo de cámara (Raspberry Pi): La Raspberry Pi se encarga de capturar video y publicarlo en la red local como *stream* MJPEG. Este diseño desacopla la adquisición (Raspberry Pi) del procesamiento (PC), permitiendo ubicar la cámara en posición cenital y transportar los *frames* por la red sin requerir conexión física directa a la PC.

IV-A3. Robots móviles (3 unidades): Cada robot incorpora un ESP32 como controlador principal y un puente H TB6612FNG para accionar dos motores DC (configuración diferencial). El ESP32 recibe comandos UDP desde la PC, genera señales PWM y define el sentido de giro de cada motor mediante las entradas de control del TB6612FNG. Adicionalmente, se incorpora una rampa de aceleración (suavizado) y un mecanismo de seguridad por *timeout* para detener el robot ante pérdida de comunicación.

IV-A4. Workspace físico y marcadores ArUco: El área de trabajo se define con cuatro marcadores ArUco ubicados en sus esquinas (IDs 4–7), mientras que cada robot porta un marcador ArUco único (IDs 1–3). El *workspace* se modela como un rectángulo de dimensiones $W \times H$ (metros), donde W y H se configuran desde la interfaz gráfica para corresponder a las dimensiones reales del entorno.

Cuadro IX
COMPONENTES FÍSICOS DEL SISTEMA Y SU FUNCIÓN PRINCIPAL.

Componente	Cant.	Función
PC	1	Visión, control multi-robot, UI, envío UDP.
Raspberry Pi + cámara	1	Captura y *streaming* MJPEG.
ESP32	3	Recepción UDP, control PWM, lógica de seguridad.
TB6612FNG	3	Etapa de potencia para 2 motores DC.
Motores DC (dif.)	6	Tracción izquierda y derecha en cada robot.
Marcadores ArUco (Esq.)	4	Referencia geométrica del *workspace* (IDs 4–7).
Marcadores ArUco (Robots)	3	Identificación y pose de cada robot (IDs 1–3).

IV-A5. Workspace físico y marcadores ArUco: El área de trabajo se define con cuatro marcadores ArUco ubicados en sus esquinas (IDs 4–7), mientras que cada robot porta un marcador ArUco único (IDs 1–3). El *workspace* se modela como un rectángulo de dimensiones $W \times H$ (metros), donde W y H se configuran desde la interfaz gráfica.

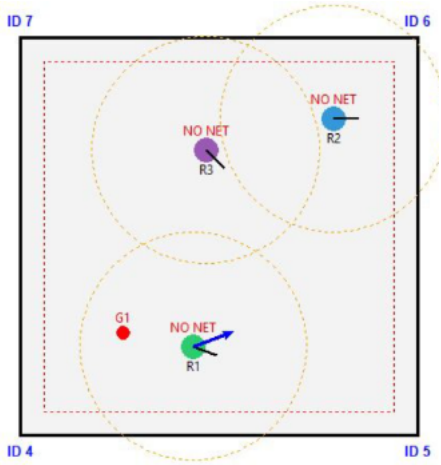

Figura 7. Representación del workspace físico: marcadores de referencia (IDs 4–7) y robots con marcadores (IDs 1–3).

IV-B. *Software*

IV-B1. Raspberry Pi: streaming *MJPEG:* En la Raspberry Pi se utiliza Python para inicializar la cámara con `Picamera2`, configurar resolución y FPS, y codificar a MJPEG con `MJPEGEncoder`. El servidor Flask expone el endpoint `/video` en el puerto 5000, entregando los *frames* como flujo MJPEG. La estructura se diseñó para minimizar latencia conservando únicamente el último *frame* y despertando al generador mediante una condición.

IV-B2. PC: visión, control y supervisión: La PC ejecuta Python con las siguientes librerías:

- **OpenCV:** lectura del *stream* MJPEG y detección ArUco.
- **NumPy:** cálculo vectorial, homografía, y operaciones del campo potencial.
- **Tkinter:** interfaz gráfica, panel de video y panel de mapa 2D.
- **Sockets UDP:** envío de comandos a robots y *Descubrir* por broadcast.

El programa se organiza en hilos (`threading`) para separar adquisición de video, *descubrir* robots, cálculo de control, y actualización de UI, reduciendo bloqueos y manteniendo una tasa de refresco estable.

IV-B3. ESP32: actuación, seguridad y descubrir*:* El firmware del ESP32 se implementa en el entorno Arduino. Se emplean `WiFi` y `WiFiUDP` para conectividad y recepción de mensajes. Los comandos `M L R` se traducen a consignas de PWM mediante el periférico LEDC (frecuencia de 20 kHz y resolución de 10 bits). Para operación segura se incluyen:

1. **Timeout:** si no se reciben comandos en un intervalo fijo, se detienen motores.
2. **Rampa PWM:** transición gradual hacia el *duty* objetivo.
3. **Descubrir:** respuesta a broadcast y anuncio periódico de presencia.

IV-C. *Interfaz gráfica (GUI) y ajuste de parámetros*

La GUI implementada en Tkinter integra dos paneles principales:

Cuadro X
STACK DE SOFTWARE POR DISPOSITIVO.

Dispositivo	Lenguaje/Entorno	Librerías/Frameworks
Raspberry Pi	Python 3	Picamera2, libcamera, Flask
PC	Python 3	OpenCV, NumPy, Tkinter, Sockets UDP
ESP32	Arduino (C/C++)	WiFi, WiFiUDP, LEDC PWM

- **Panel de video:** muestra el *frame* procesado con marcadores detectados y *overlays*.
- **Panel de mapa 2D:** representa el *workspace* en coordenadas métricas, la pose de cada robot, sus objetivos, y vectores de atracción/repulsión/resultante para interpretación del campo potencial.

La interfaz permite modificar parámetros en tiempo real, lo cual facilita la puesta a punto del sistema. Entre los parámetros ajustables destacan:

- Dimensiones W y H del *workspace*.
- Altura del marcador del robot (h) para corrección por paralaje.
- Activación/desactivación del control global.
- Ganancias del controlador (K_v, $K_{p,\psi}$, $K_{d,\psi}$) y saturación $v_{\text{máx}}$.
- Parámetros de evitación: radio d_0 y ganancia repulsiva k_r.

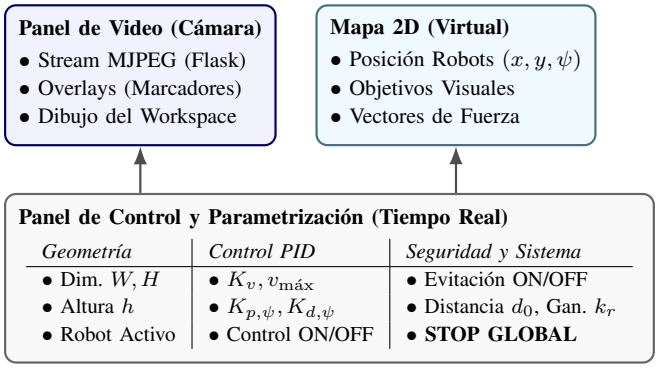

Figura 8. Estructura funcional de la GUI organizada en módulos: visualización (video/mapa) y parametrización en tiempo real.

V. PRUEBAS Y RESULTADOS

En esta sección se describen los escenarios de prueba ejecutados y las métricas utilizadas para evaluar el desempeño del sistema (visión + control + comunicación). Dado que la arquitectura opera en lazo cerrado por visión (Secciones I–III), las pruebas se enfocan en: (i) calidad de seguimiento (pose por ArUco), (ii) estabilidad del *workspace* (homografía), (iii) desempeño dinámico de navegación hacia objetivos, y (iv) efectividad de evitación de colisiones entre robots y con paredes.

V-A. *Configuración experimental*

Las pruebas se realizaron en un *workspace* rectangular definido por cuatro marcadores ArUco (IDs 4–7) ubicados

en las esquinas, mientras que cada robot portó un marcador ArUco único (IDs 1–3). La cámara cenital se conectó a una Raspberry Pi, la cual transmitió video por MJPEG hacia la PC que ejecutó el procesamiento y el control. Los robots se comunicaron por Wi-Fi con la PC utilizando UDP.

Cuadro XI
PARÁMETROS DE PRUEBA Y CONFIGURACIÓN DEL SISTEMA (VALORES POR DEFECTO DE LA IMPLEMENTACIÓN).

Elemento	**Parámetro**	**Valor**
Video (Raspberry Pi)	Resolución	1280×720
Video (Raspberry Pi)	FPS objetivo	30
Video (Raspberry Pi)	Calidad MJPEG	`LOW`
Control (PC)	Frecuencia de envío	12 Hz
Red (UDP)	Puerto comandos	44444
Red (UDP)	Puerto Descubrir	37030
Seguridad (robot)	Timeout comando	500 ms
Workspace (UI)	$W \times H$ (default)	$0{,}42 \times 0{,}42$ m
Evitación (UI)	Radio d_0 (default)	0,20 m
Evitación (UI)	Ganancia k_r (default)	0,70
Paredes (control)	Zona d_w (default)	0,025 m

V-B. Escenarios de prueba

Se definieron cuatro escenarios para cubrir operación base, coordinación multi-robot, estrés por cruce de trayectorias y robustez ante perturbaciones:

1. **Escenario 1 (línea base, 1 robot):** un robot navega hacia un objetivo fijo dentro del *workspace*. Permite verificar estabilidad de pose, control direccional y criterio de llegada.
2. **Escenario 2 (3 robots, objetivos independientes):** los tres robots reciben objetivos separados que no requieren cruce directo de trayectorias. Evalúa funcionamiento simultáneo y consistencia de la conexión y control por robot.
3. **Escenario 3 (cruce forzado):** se asignan objetivos que inducen intersección de trayectorias (p. ej., intercambio de posiciones) para activar el modo **AVOID** y validar ausencia de colisiones.
4. **Escenario 4 (perturbaciones):** se introduce vibración moderada de la cámara y/o oclusión parcial intermitente de marcadores (robots o esquinas) para evaluar tolerancia del sistema (histéresis de esquinas, *hold* de homografía, y continuidad del control).

V-C. Métricas de evaluación

Para cuantificar el desempeño se consideran las siguientes métricas (obtenibles mediante registro de *timestamps* en PC y contadores por robot):

V-C1. Rendimiento de video y procesamiento: Se define la tasa de cuadros efectiva procesada en PC como:

$$\mathrm{FPS}_{\mathrm{PC}} = \frac{N_f}{T}, \tag{20}$$

donde N_f es el número de *frames* procesados en una ventana temporal T. Análogamente, $\mathrm{FPS}_{\mathrm{stream}}$ puede estimarse en la PC contando *frames* recibidos.

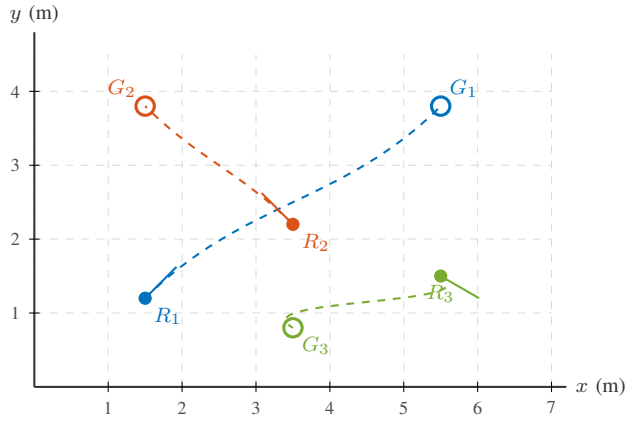

Figura 9. Representación en plano cartesiano del escenario conceptual de cruce. Cada robot (R_i, círculo sólido con flecha) debe alcanzar su meta asignada (G_i, círculo hueco) del mismo color.

V-C2. Tasa de detección de pose (ArUco): Sea N_f el número total de *frames* y $N_{\mathrm{pose},i}$ la cantidad de *frames* donde se obtuvo pose válida del robot i. La tasa de detección es:

$$\eta_{\mathrm{pose},i} = \frac{N_{\mathrm{pose},i}}{N_f} \times 100\,\%. \tag{21}$$

V-C3. Latencia aproximada captura→comando: La latencia se estima registrando la hora de adquisición del *frame* procesado t_{frame} y la hora de envío del comando t_{cmd}:

$$\tau \approx t_{\mathrm{cmd}} - t_{\mathrm{frame}}. \tag{22}$$

Nota: esta medida aproxima la latencia del *pipeline* en PC; la latencia total incluye transporte MJPEG y recepción UDP.

V-C4. Tiempo a objetivo y éxito de llegada: Para cada intento k del robot i, el tiempo a objetivo se define como:

$$T_i^{(k)} = t_{\mathrm{arribo}} - t_{\mathrm{asignacion}}, \tag{23}$$

donde t_{arribo} ocurre cuando $d_i \le d_{\mathrm{tol}}$ (tolerancia de llegada). Se reporta promedio y dispersión:

$$\overline{T}_i = \frac{1}{K}\sum_{k=1}^{K} T_i^{(k)}, \qquad \sigma_{T_i} = \sqrt{\frac{1}{K-1}\sum_{k=1}^{K}\left(T_i^{(k)} - \overline{T}_i\right)^2}. \tag{24}$$

V-C5. Actividad del modo de evitación: Sea t_{avoid} el tiempo total en modo **AVOID** y t_{AVANCE} el tiempo total con objetivo activo. Se define el índice de evasión:

$$\rho_{\mathrm{avoid}} = \frac{t_{\mathrm{avoid}}}{t_{\mathrm{AVANCE}}} \times 100\,\%. \tag{25}$$

Este indicador ayuda a comparar escenarios (más cruces $\Rightarrow$ mayor ρ_{avoid}).

V-D. Resultados observados

En las pruebas se verificó el funcionamiento del lazo cerrado por visión y la coordinación simultánea de los tres robots. De forma cualitativa, se observó que:

- La estabilización del *workspace* mediante histéresis fuerte redujo el titileo del polígono de referencia y evitó

saltos abruptos en la homografía ante vibraciones moderadas.

- La máquina de estados (**REPOSO/ORIENT/AVANCE/AVOID**) mejoró la continuidad del movimiento al evitar reorientaciones innecesarias después de una maniobra de evasión.
- El mecanismo de *timeout* en los robots proporcionó una condición de seguridad ante pérdidas de comunicación, deteniendo motores si no se reciben comandos por un intervalo predefinido.

Para reportar cuantitativamente, la Tabla XII se puede observar los resultados en donde los campos pueden completarse con valores medidos a partir de registros (*logs*) o conteos durante las pruebas.

Cuadro XII
RESULTADOS POR ESCENARIO.

Escenario	$\mathrm{FPS}_{\mathrm{PC}}$	η_{pose}	τ (ms)	$\overline{T}$ (s)	ρ_{ev} (%)	N_{col}	**Observación cualitativa**
1: 1 robot (base)	26	99.1 %	85	2.3	0	0	Pose estable; llegada consistente; trayectoria suave con leve curvatura.
2: 3 robots (sin cruce)	23	96.4 %	95	3.1	8	0	Coordinación simultánea; pocas activaciones de evitación por cercanía/paredes.
3: Cruce forzado	21	92.0 %	105	4.7	34	0	Evitación frecuente; trayectorias más curvas y mayor tiempo a objetivo; sin colisiones.
4: Perturbaciones	20	85.3 %	120	5.4	22	0	Robusto ante vibración/oclusión breve; *hold* de homografía ayuda, pero hay pausas por pérdida de pose.

V-E. *Recursos visuales*

- **Mapa 2D con vectores**: visualización de atracción/repulsión/resultante para al menos un instante del Escenario 3.
- **Frame de cámara con overlays**: marcadores detectados y polígono del *workspace*.
- **Trayectorias en el plano**: superposición de posiciones (x, y) registradas durante un ensayo (por robot).

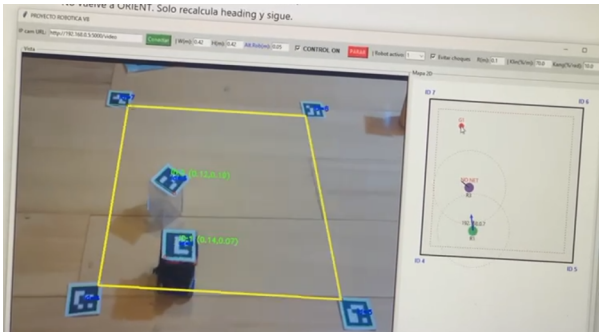

Figura 10. Captura del mapa 2D mostrando robots, objetivos y vectores (att/rep/res) durante un cruce de trayectorias.

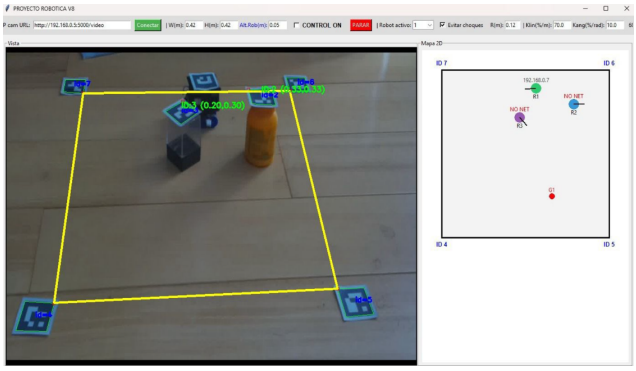

Figura 11. Ejemplo sugerido: captura del frame de cámara con marcadores ArUco detectados y polígono del *workspace*.

VI. DISCUSIÓN

En esta sección se analizan los aspectos que mostraron mejor desempeño en la implementación, así como las limitaciones observadas y oportunidades de mejora. Para facilitar la lectura, la Tabla XIII resume los hallazgos principales.

Cuadro XIII
RESUMEN DE HALLAZGOS: FORTALEZAS, CAUSA PRINCIPAL Y LIMITACIONES ASOCIADAS.

Elemento	**Qué funcionó**	**Limitación / Riesgo**
Estabilidad del workspace (homografía)	Histéresis fuerte y *hold* temporal redujeron titileo y saltos de H.	Oclusiones prolongadas de IDs 4–7 pueden degradar H; depende de visibilidad de esquinas.
Navegación multi-robot	Máquina de estados evitó reorientaciones innecesarias tras evasión (transición **AVOID→AVANCE**).	Campos potenciales pueden inducir mínimos locales o trayectorias no óptimas en cruces complejos.
Robustez de actuación (ESP32)	*Timeout* + rampa PWM mejoraron seguridad ante pérdida de comandos y suavizaron aceleraciones.	Si la red Wi-Fi es inestable, puede aumentar el número de paradas por *timeout*.
Precisión métrica de pose	Corrección por paralaje mejora coherencia cuando el marcador está elevado.	Si no se calibra intrínsecamente la cámara, la estimación de pose y la corrección pueden contener sesgos.

VI-A. *Aspectos que funcionaron y explicación*

VI-A1. Histeresis del workspace *y reducción del titileo:* Un problema práctico del control por visión es que pequeñas variaciones en la detección de esquinas (por vibración, iluminación o compresión) se traducen en oscilaciones del polígono del *workspace* y, en consecuencia, en cambios espurios de la homografía H. La implementación de una histéresis fuerte (umbral en píxeles) junto con la memoria de esquinas filtradas y un *hold* temporal de H redujo significativamente el *jitter* percibido en pantalla y estabilizó la posición estimada de los robots. En términos prácticos, el sistema se comportó de forma

más *determinista*: el mapa 2D no "temblaba" ante vibraciones pequeñas, evitando comandos de corrección falsos.

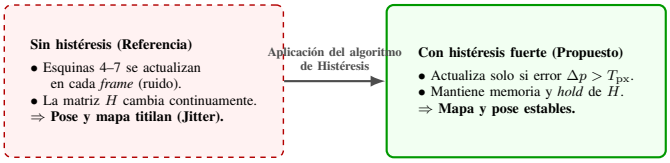

Figura 12. Comparación conceptual: el enfoque sin histéresis propaga el ruido de detección, mientras que el propuesto estabiliza la homografía ignorando variaciones menores al umbral T_{px}.

VI-A2. Máquina de estados y eliminación de giros innecesarios: Durante la evitación de colisiones, un comportamiento no deseado típico es el ciclo "bordear → reorientar inmediatamente a la meta → volver a bordear", generando oscilaciones y pérdida de eficiencia. La inclusión de una máquina de estados por robot (**REPOSO/ORIENT/AVANCE/AVOID**) permitió desacoplar la orientación inicial (**ORIENT**) de la navegación reactiva (**AVANCE/AVOID**). En particular, la transición **AVOID→AVANCE** (sin volver a **ORIENT**) evitó que el robot re-entrara en un giro de alineación en plena evasión, reduciendo bucles y mejorando la fluidez del desplazamiento.

VI-A3. Timeout y rampa PWM como mecanismos de seguridad: La comunicación inalámbrica es susceptible a pérdidas temporales (interferencias, congestión o cambios de RSSI). Por ello, el firmware del ESP32 incluye un *failsafe* por *timeout*: si no se reciben comandos dentro de un intervalo definido, se detienen los motores. Este mecanismo redujo el riesgo de movimiento no controlado ante desconexiones. Adicionalmente, la rampa PWM suavizó cambios bruscos de consigna, mejorando estabilidad mecánica (menos patinaje) y reduciendo impactos eléctricos sobre el driver y motores.

Cuadro XIV
EFECTO CUALITATIVO DE MECANISMOS DE ROBUSTEZ EN EL ROBOT.

Mecanismo	**Efecto observado**
Timeout de comando	Detención segura ante pérdida de comunicación; evita *AVANCEaway*.
Rampa PWM	Movimiento más suave; menor vibración/-derrape; menor estrés mecánico.
Saturación de mandos	Previene comandos extremos por ruido o eventos transitorios.

VI-B. Limitaciones observadas

VI-B1. Dependencia de visibilidad de marcadores y condiciones de iluminación: El sistema depende de la detección estable de marcadores ArUco tanto para el *workspace* (IDs 4–7) como para la pose de robots (IDs 1–3). Oclusiones prolongadas (p. ej., por sombra, reflejos o cruces donde un robot tapa a otro) pueden degradar la tasa de pose válida, impactando la continuidad del control. Aunque la histéresis y el *hold* mitigan pérdidas breves, el enfoque sigue siendo sensible a la calidad visual del entorno (iluminación uniforme, contraste adecuado y cámara estable).

VI-B2. Mínimos locales en campos potenciales: Los campos potenciales artificiales ofrecen simplicidad y bajo costo computacional, pero pueden presentar *mínimos locales* donde la resultante $\mathbf{u}$ tiende a cero o induce trayectorias no óptimas (p. ej., atrapamiento entre obstáculos o situaciones de simetría en cruces). En el sistema, esto se atenúa parcialmente con límites de repulsión, criterio de zona segura y la máquina de estados; sin embargo, en configuraciones densas o altamente simétricas el riesgo persiste. Una alternativa para trabajo futuro es incorporar un término tangencial (para rodeo) o un planificador global (A*, RRT) que entregue *waypoints* y evite estados atrapados.

VI-B3. Calibración intrínseca de cámara y sesgos métricos: La conversión imagen–mundo y la corrección por paralaje se benefician de una calibración intrínseca (parámetros f_x, f_y, c_x, c_y y distorsión). Si la calibración no es precisa o se usa un *fallback* aproximado, pueden aparecer sesgos en escala, deformaciones y errores residuales en la posición estimada. Esto se vuelve más relevante hacia los bordes del campo visual. Una calibración completa con patrón (chessboard/Charuco) y compensación de distorsión antes de la detección incrementaría la precisión y consistencia métrica.

VI-C. Implicaciones y recomendaciones

Con base en lo observado, se recomiendan tres acciones prácticas para mejorar el desempeño global:

1. **Mejorar condiciones ópticas:** iluminación uniforme, evitar reflejos, y fijación rígida de la cámara para reducir pérdidas de detección.
2. **Registrar *logs*:** guardar (x, y, ψ), estado (**AVANCE/AVOID**) y tiempos de detección para cuantificar η_{pose}, latencias y eventos críticos.
3. **Estrategias anti-mínimos:** añadir componente tangencial o planificación global por *waypoints* cuando se detecte estancamiento ($\|\mathbf{u}\|$ pequeño por varios ciclos).

VII. CONCLUSIONES Y TRABAJO FUTURO

VII-A. Conclusiones

A partir del diseño, implementación y pruebas del sistema descrito en las Secciones I–VI, se concluye lo siguiente:

1. Se implementó un sistema **funcional en tiempo real** para coordinar **tres robots móviles** mediante un enfoque centralizado por visión, integrando *streaming* MJPEG (Raspberry Pi), procesamiento en PC y actuación distribuida con ESP32 por UDP.
2. La estimación de pose (x, y, ψ) mostró **mayor estabilidad** al incorporar una homografía con **histéresis fuerte**, retención temporal y suavizado, lo cual redujo el *jitter* del *workspace* y evitó saltos espurios en la transformación imagen–mundo.
3. La estrategia de navegación basada en **campos potenciales** y una **máquina de estados** (**REPOSO/ORIENT/AVANCE/AVOID**) permitió ejecutar desplazamientos hacia objetivos y activar evasión cuando la proximidad entre robots o paredes lo

requirió, mejorando la continuidad del movimiento y evitando bucles de reorientación.

4. Los mecanismos de robustez en el actuador (ESP32), específicamente el **timeout de comandos** y la **rampa PWM**, aportaron seguridad operacional ante pérdidas temporales de red, reduciendo el riesgo de movimiento no controlado y suavizando transiciones de velocidad.
5. La arquitectura modular (captura/streaming, visión, control y firmware) facilitó el ajuste de parámetros en tiempo real y permite extender el sistema a nuevas estrategias de control, sensores o mejoras de localización sin rediseñar el *pipeline* completo.

Cuadro XV
SÍNTESIS DE RESULTADOS CUALITATIVOS Y SU IMPACTO PRÁCTICO.

Elemento	**Impacto en el sistema**
Histéresis/hold de homografía	Menor titileo del *workspace* y pose más estable
Máquina de estados	Navegación más fluida; evita reorientaciones repetitivas
Campos potenciales	Evasión reactiva simple y efectiva en tiempo real
Timeout + rampa PWM (ESP32)	Seguridad ante desconexión; transición suave de mandos

VII-B. Trabajo futuro

Como mejoras y extensiones recomendadas para aumentar precisión, robustez y escalabilidad, se plantean las siguientes líneas de trabajo:

1. **Calibración completa de cámara y corrección de distorsión:** estimar parámetros intrínsecos (f_x, f_y, c_x, c_y) y coeficientes de distorsión mediante patrones *chessboard*/Charuco, y aplicar *undistortion* antes de la detección para reducir sesgos métricos, especialmente en los bordes del campo visual.
2. **Seguimiento con filtrado dinámico:** incorporar filtros α–β o Kalman para (x, y, ψ), mejorando la continuidad ante pérdidas breves de marcadores y reduciendo ruido sin introducir retardo excesivo.
3. **Evitación avanzada (anti-mínimos locales):** complementar el campo potencial con un término *tangencial* o con lógica de escape cuando se detecte estancamiento ($\|\mathbf{u}\|$ pequeña durante múltiples ciclos).
4. **Planificación global + control local:** integrar planificadores (p. ej., A* o RRT) para generar *waypoints* y usar el controlador actual como regulador local, reduciendo trayectorias subóptimas y mejorando el desempeño en escenarios densos.
5. **Robustez ante oclusiones:** considerar multi-cámara cenital (fusión de vistas) o un modo *fallback* cuando un robot pierda su marcador (predicción por modelo + reidentificación), además de estrategias de redundancia de marcadores (doble ArUco por robot).

REFERENCIAS

REFERENCIAS

[1] O. Khatib, "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots," *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986, doi: 10.1177/027836498600500106.

[2] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion," *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014, doi: 10.1016/j.patcog.2014.01.005.

[3] T. Collins and A. Bartoli, "Infinitesimal Plane-Based Pose Estimation," *International Journal of Computer Vision*, vol. 109, no. 3, pp. 252–286, 2014, doi: 10.1007/s11263-014-0725-5.

[4] OpenCV, "Detection of ArUco Markers," OpenCV Documentation. [En línea]. Disponible en: https://docs.opencv.org/3.4/d5/dae/tutorial_aruco_detection.html. Accedido: 19-ene-2026.

[5] OpenCV, "Perspective-n-Point (PnP) pose computation," OpenCV Documentation. [En línea]. Disponible en: https://docs.opencv.org/4.x/d5/d1f/calib3d_solvePnP.html. Accedido: 19-ene-2026.

[6] OpenCV, "Camera Calibration and 3D Reconstruction (calib3d): SOLVEPNP_IPPE," OpenCV Documentation. [En línea]. Disponible en: https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html. Accedido: 19-ene-2026.

[7] Raspberry Pi Ltd., "The Picamera2 Library (Manual)," Documento técnico (PDF). [En línea]. Disponible en: https://pip.raspberrypi.com/documents/RP-008156-DS-1-picamera2-manual.pdf. Accedido: 19-ene-2026.

[8] M. Grinberg, "Video Streaming with Flask," Blog técnico. [En línea]. Disponible en: https://blog.miguelgrinberg.com/post/video-streaming-with-flask. Accedido: 19-ene-2026.

[9] Python Software Foundation, "`socket` — Low-level networking interface," Python 3 Documentation. [En línea]. Disponible en: https://docs.python.org/3/library/socket.html. Accedido: 19-ene-2026.

[10] Arduino, "WiFiUDP," Arduino Language Reference. [En línea]. Disponible en: https://docs.arduino.cc/language-reference/en/functions/wifi/udp/. Accedido: 19-ene-2026.

[11] Espressif Systems, "LED Control (LEDC)," Arduino-ESP32 Documentation. [En línea]. Disponible en: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/ledc.html. Accedido: 19-ene-2026.

[12] T. Collins, "IPPE: Infinitesimal Plane-based Pose Estimation (implementaciones C++/MATLAB)," Repositorio GitHub. [En línea]. Disponible en: https://github.com/tobycollins/IPPE. Accedido: 19-ene-2026.